JavaScript Basics - Assignment 09

ES6+ Features, Destructuring, Spread Operator & Modules

Topics Covered: Array Destructuring, Object Destructuring, Default Values, Renaming Properties, Nested Objects, Spread Operator, Copying and Merging Data, Named Exports, Default Exports, Imports

Important Instructions

- Read each scenario carefully and plan your solution before writing code.
- Create a **separate folder or clearly separated set of files for each question**. Questions involving modules will require more than one file.
- Use clean, properly indented code and meaningful variable and file names.
- Use only JavaScript concepts taught in class. Do not use frameworks or external libraries.
- Display results using **console.log()** unless a question asks for a webpage display.
- For module-based questions, load the main JavaScript file with **type="module"** and run the webpage through a local development server such as VS Code Live Server.
- Test your code before submission. Be prepared to explain how your solution works.

Submission Instructions

- Submit your assignment through a **link to your GitHub repository**.
- Keep each question separate and include every HTML, CSS, and JavaScript file required for that question.
- For module questions, keep the imported and exported files together and use correct relative file paths.
- Push all final files to GitHub before submitting. Ensure the repository is accessible to the instructor.

Question 1 - Library Book Record

A library system receives book information from an older system as an ordered list. Use the sample record below and prepare the information needed for a book details card.

Sample record: **["BK-104", "JavaScript for Beginners", "Programming", 1200]**

- Extract the book ID, book title, and price into separate variables.
- Ignore the category value when extracting the details in one statement.
- The system later receives a shorter record: **["BK-105", "Learning ES6"]**. Prepare a category variable that uses **"General"** when the record does not include a category.
- Display the extracted details in a readable sentence using a template literal.
- Do not change the original arrays.

Question 2 - Student Profile from an API

A school website receives the following student profile from an API. Prepare the values required for a profile summary without repeatedly accessing each property.

Sample profile: **{ name: "Sara Ahmed", age: 19, department: "Software Engineering", address: { city: "Karachi", country: "Pakistan" } }**

- Extract the student's name and age into variables.
- Store the department in a local variable named **program**, while keeping the original property name unchanged in the object.
- Extract the city and country from the address information.
- The API may sometimes omit the student phone number. Prepare a phone variable that uses **"Not provided"** when no phone value is available.
- Display a short profile summary containing the student's name, program, city, and phone value.

Question 3 - Manage Class Lists and Student Profiles

You are updating a school's class management page. The school has two groups of students and wants to prepare updated data without accidentally changing the original lists or profile.

Group A: **["Ali", "Ahmed", "Hamza"]**
Group B: **["Sara", "Ayesha", "Laiba"]**
Profile: **{ name: "Laiba", className: "Web Development", city: "Karachi" }**

- Create a combined class list containing the students from both groups.
- Create an independent copy of Group A. Add a new student to the copy and show that the original Group A has not changed.
- Create an updated profile for Laiba in which the city is changed to **"Lahore"** while the original profile remains unchanged.
- Create another updated profile that keeps the existing details and adds a new property called **email**.
- Display the original data and updated data so the differences are clear.

Question 4 - Build a Reusable Student Toolkit with Modules

A school website has become difficult to maintain because all student-related code is written in one file. Separate the reusable student functions from the main page logic.

Create these files: `index.html`, `studentTools.js`, and `main.js`.

- In **studentTools.js**, create and make available a function that receives a student's first and last name and returns the full name.
- In the same file, create and make available a function that receives marks from four subjects and returns the total marks.
- Make one of these functions available as a named export and the other as the file's default export.
- In **main.js**, bring both functions in from **studentTools.js** and use them with sample student data.
- Display the full name and total marks in the console.
- In **index.html**, load **main.js** correctly so the browser recognizes it as a module.
- Run the page with a local development server and make sure both imported functions work.

Question 5 - Mini Project: Course Registration Data

Create a small course registration page for a training institute. The institute stores its course data in one file and uses another file to prepare the page's course information.

Start with at least three course records. Each course should contain a course name, instructor, duration, and fee.

- Store the original course records in a separate JavaScript file and make the data available to the main file.
- In the main file, prepare a summary for one course by extracting the course name and fee into variables.
- Create a separate list containing the existing courses plus one newly introduced course. Keep the original course list unchanged.
- Create an updated version of one course with a revised fee while preserving its other details and the original course object.
- Use reusable code from a separate module to format a course summary, such as a sentence containing the course name, instructor, and fee.
- Display all course summaries in the console or on the HTML page.
- Create an **index.html** file that loads your main JavaScript file as a module.
- Use correct import/export syntax and relative file paths. Test the complete project through a local development server.

Suggested Project Structure

```
Assignment-09/
  Q1_LibraryRecord.js
  Q2_StudentProfile.js
  Q3_ClassManagement.js
  Q4_StudentToolkit/
    index.html
    studentTools.js
    main.js
  Q5_CourseRegistration/
    index.html
    courses.js
    courseUtils.js
    main.js
```

Final Submission Checklist

- ■ Each question is saved separately.
- ■ Array and object destructuring tasks are completed.
- ■ Missing data and renamed properties are handled where required.
- ■ Arrays and objects are copied or combined without changing the originals where required.
- ■ Named and default exports/imports are demonstrated.
- ■ Module files use correct file paths and load through a local development server.
- ■ All files are pushed to GitHub.
- ■ GitHub repository link is ready to submit.

Analyze the data, choose the right ES6+ feature, and keep your code reusable. Happy Coding!